

Forward-Looking Statement Disclaimer

This presentation may contain forward-looking statements under applicable securities laws.

All statements other than statements of historical facts are forward-looking statement. Forward-looking statements are based on information available to Jamf at the time they are made and provide Jamf's current expectations and projections about its financial condition, results, plans, objectives, future performance and business. You can identify forward-looking statements by the fact that they do not relate strictly to historical or current facts. Forward-looking statements may include words such as "anticipate", "estimate", "project", "plan", "intend", "believe", "may", "will", "should", "can have", "likely" and other terms of similar meaning in connection with any discussion of timing or nature of future operating or financial performance or other events.

All statements we make about our estimated and projected costs, expenses, cash flows, growth rates and financial results are forward-looking statements. In addition, our plans and objectives for future operations, growth initiatives, product plans and product strategies are forward-looking statements.

There are various factors, risks and uncertainties that may cause Jamf's actual results to differ materially from those that we expected in any forward-looking statements. These factors and risks are set forth in the documents Jamf files with U.S. Securities and Exchange Commission (SEC). These documents are publicly available on the SEC's website at www.sec.gov or on Jamf's website at ir.jamf.com.

Forward-looking statements are not guarantees of future performance and you should not place undue reliance on them. Jamf is under no obligation to update any forward-looking statements made in this presentation, except as required by law.



Beyond Self Service:

Surfacing User-Friendly Information with Support App Extensions



SCOTT KENDALL

macOS Support Engineer
Giant Eagle

Focus on automation, scripting and endpoint management in enterprise environments, by creating scalable, user-friendly solutions that enhance the end-user experience and simplify IT operations.



JACOB EDWARDS

User Environment Sr. Engineer
Quad

Began working with Jamf Pro in 2015 and currently manages approximately 1700 Macs



AGENDA

1. What is Support App by root3?

2. Customization Options

3. Basic Extension Template

4. How to modify using Extensions / Showcase

5. Q&A

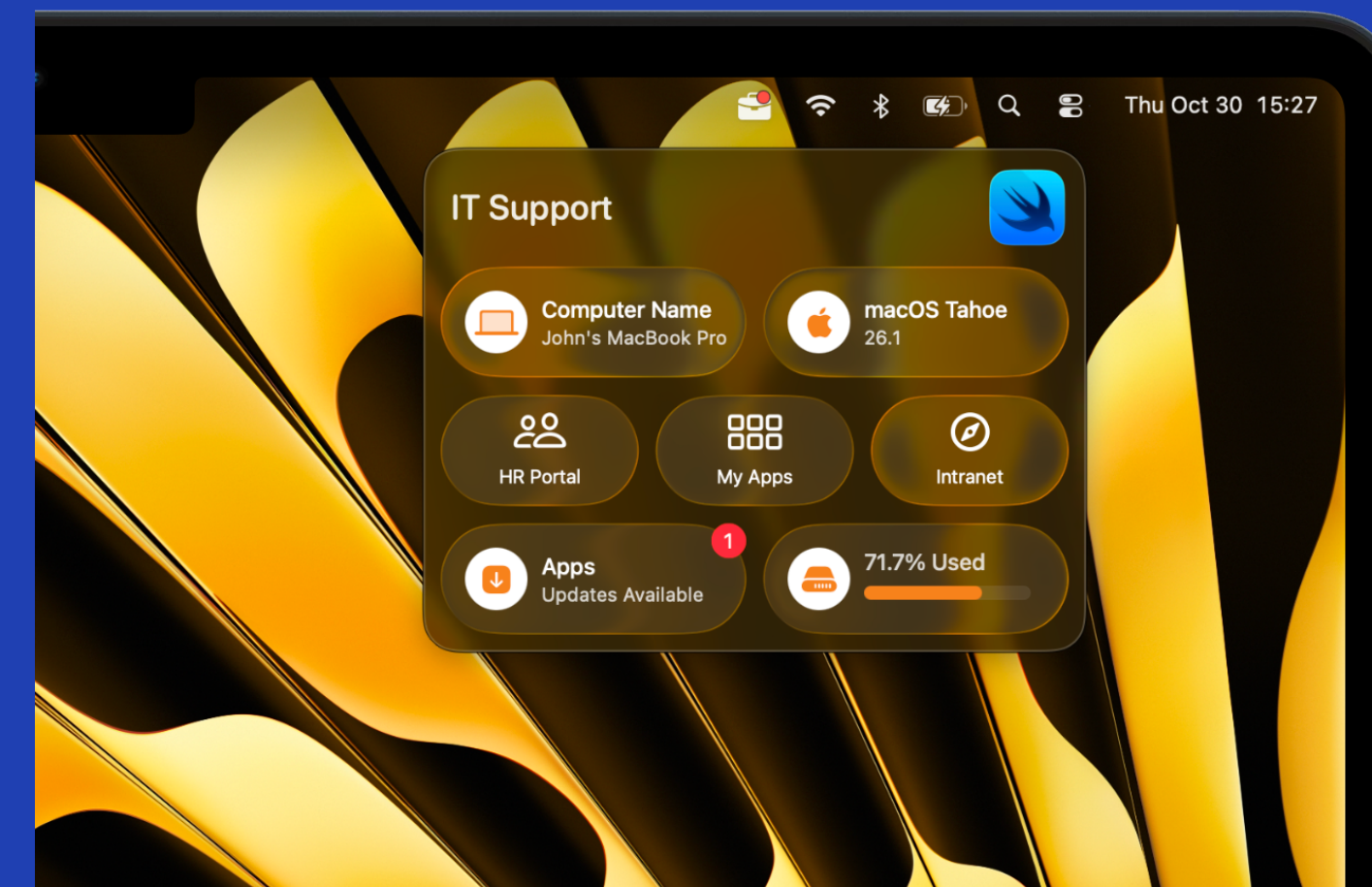
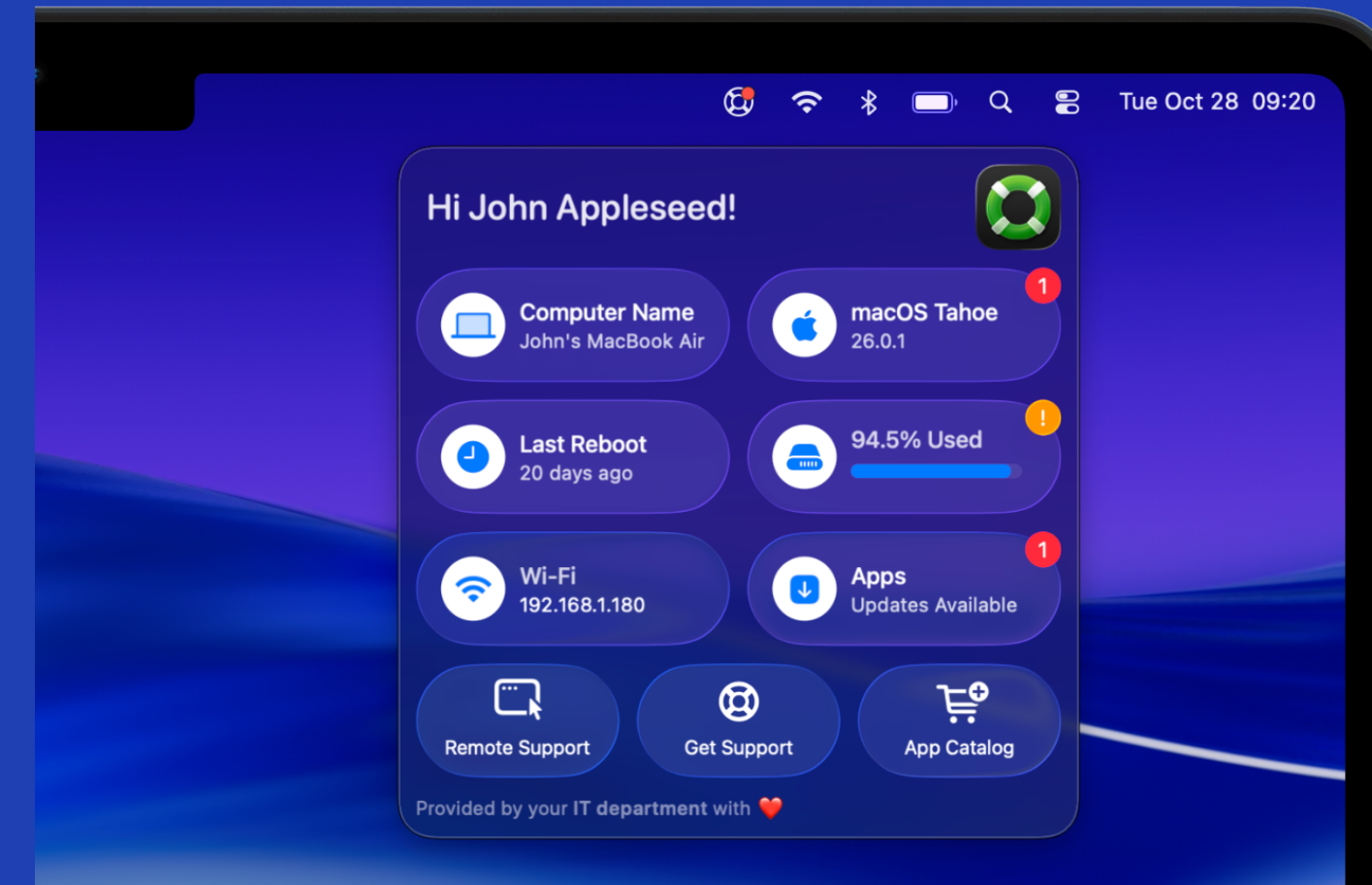


What Is **Support App** by **Root3**?

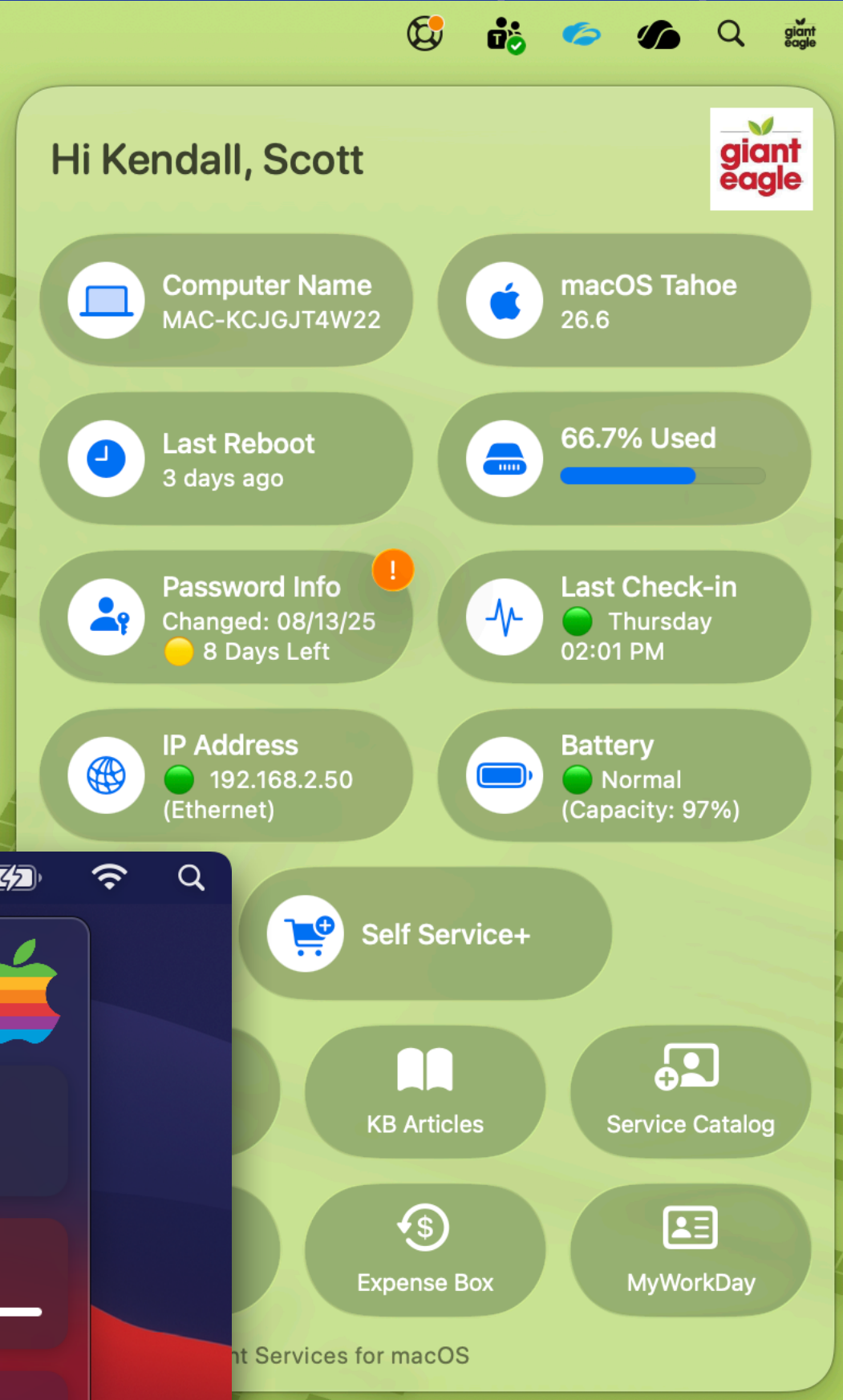
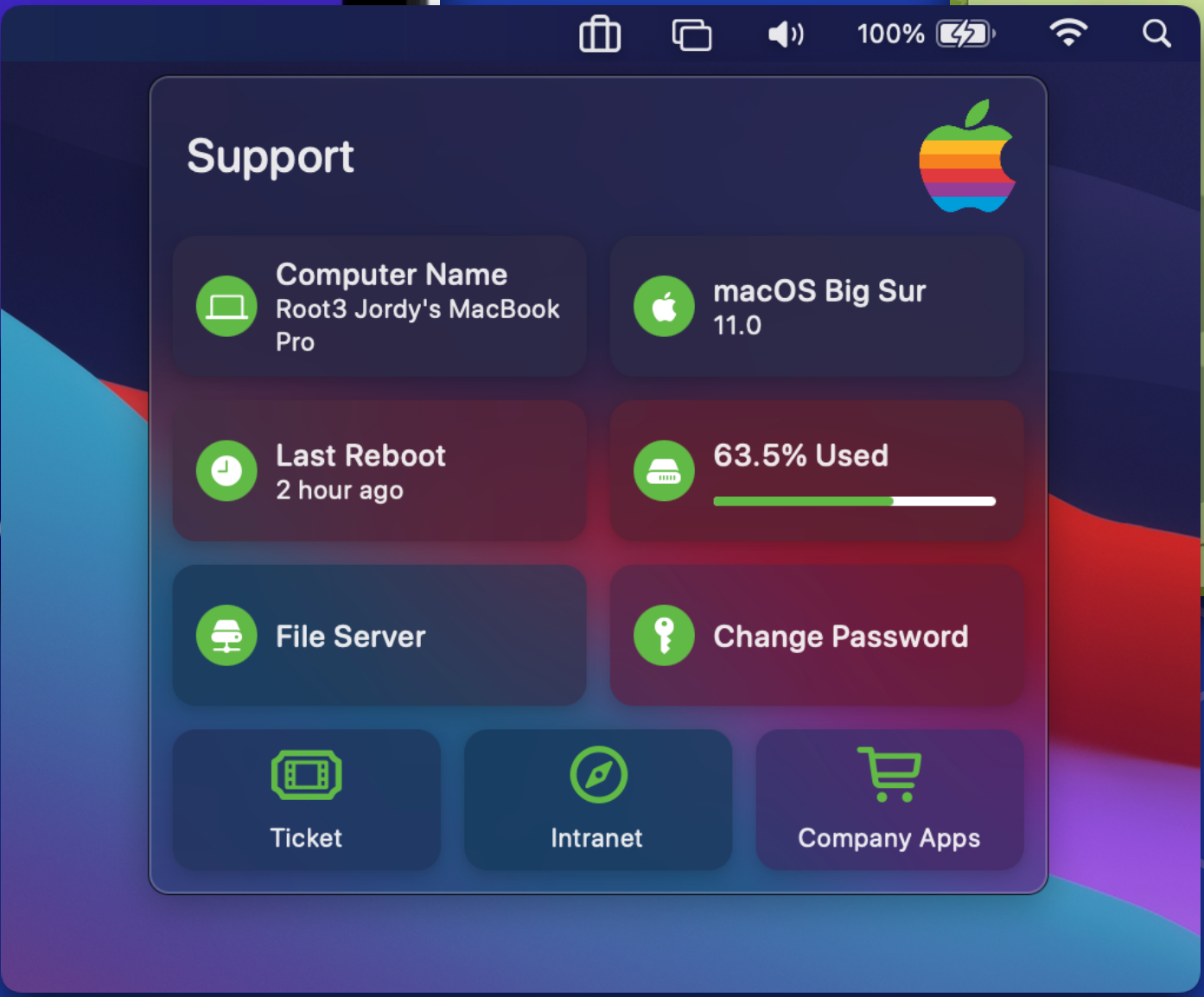
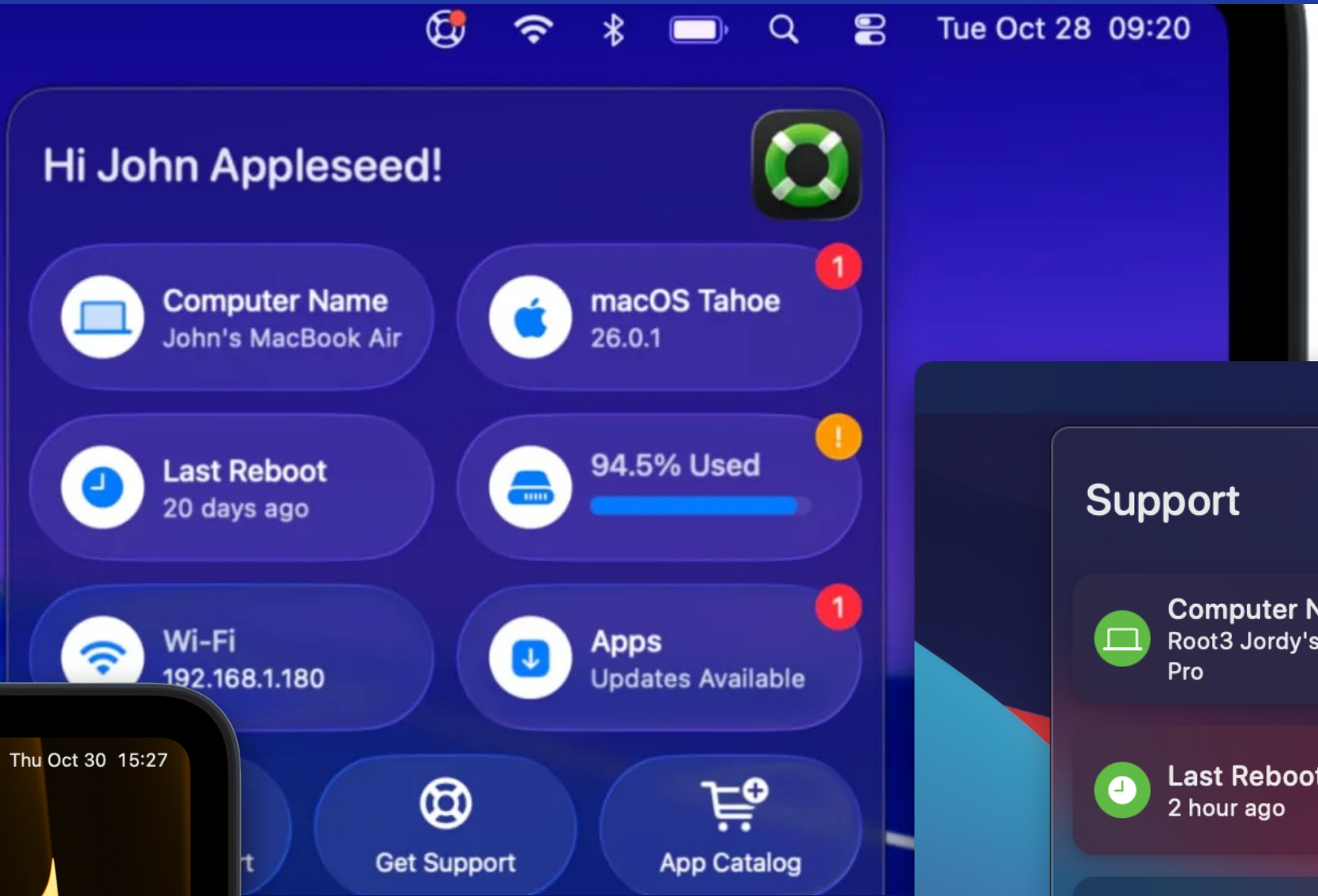
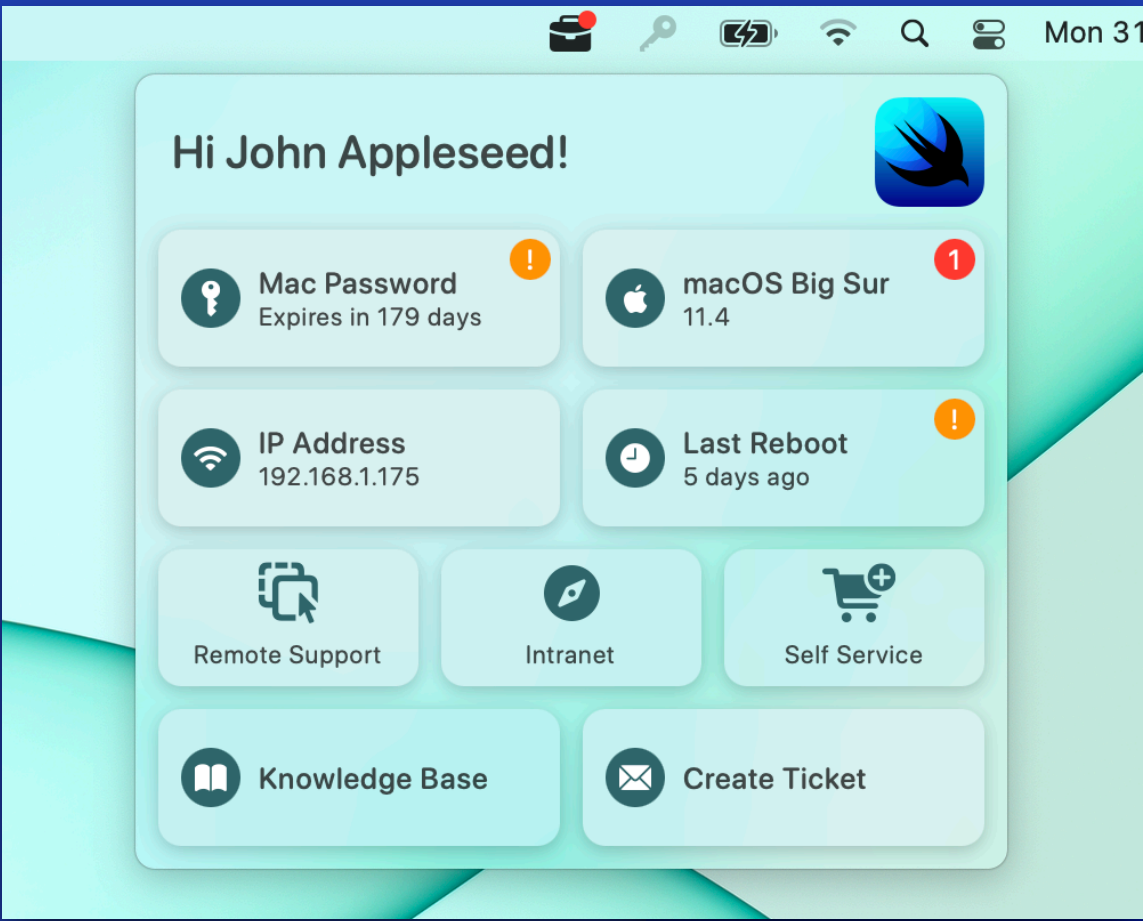


What Is **Support App**?

- ▶ Basic diagnostic information and ability to notify users
- ▶ Shortcuts to access support channels or other resources
- ▶ Modern and native macOS app for macOS 14+
- ▶ Supports your corporate identity
- ▶ Supports Dark / Light Mode
- ▶ Supports SF Symbols
- ▶ Integrates with JAMF via Config Profiles



Example Screenshots



Customization Options

Button Configurations

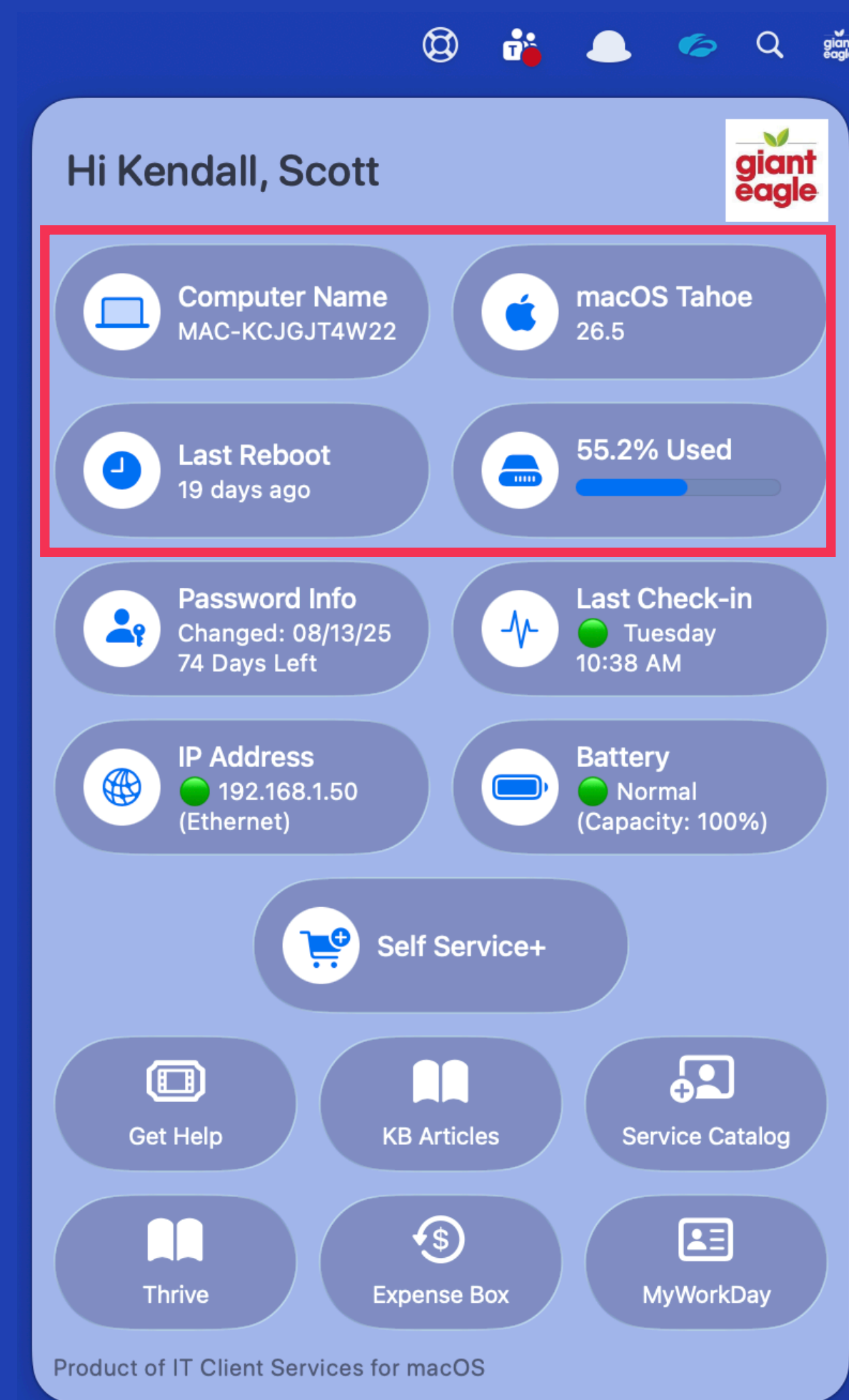


Customizing Support App

Three different types of buttons

► *Info (Diagnostic)*

- Computer Name
- OS Version
- Storage
- Uptime
- Application Updates

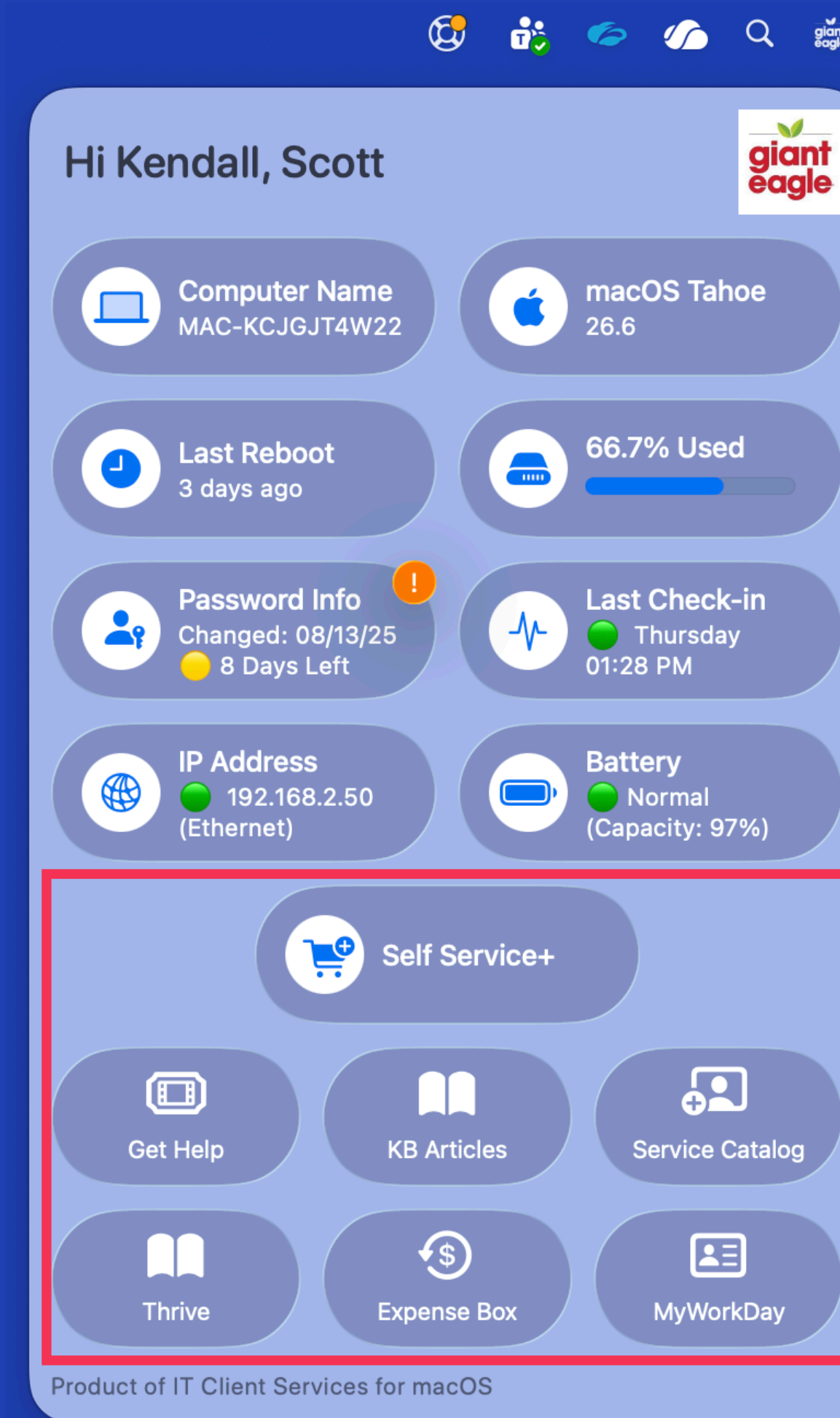


Customizing Support App

Three different types of buttons

► *Button (Sm / Med / Lg)*

- Launch Applications
- Open a URL
- Run User Commands
- Run Privileged Commands / Scripts (with SupportHelper)

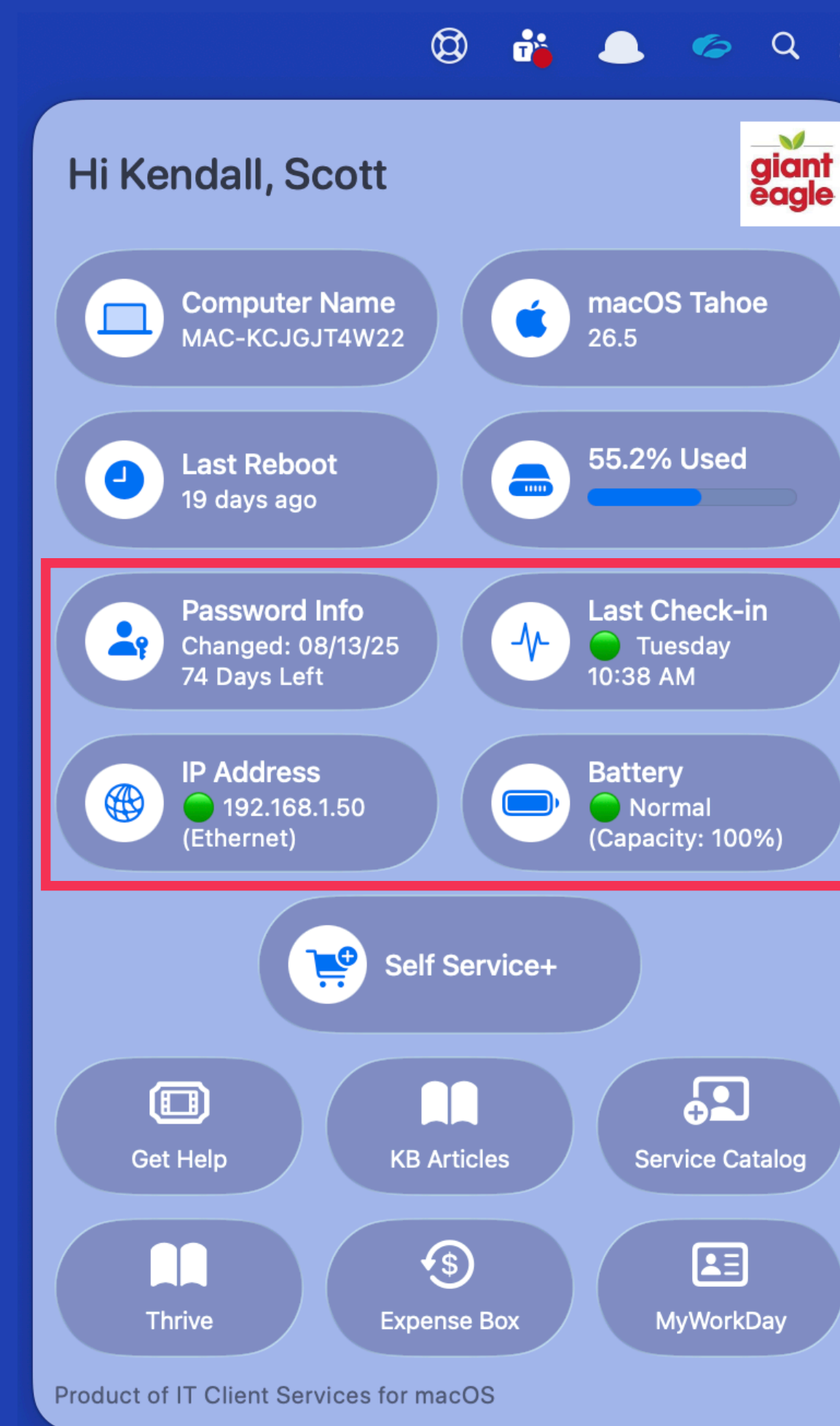


Customizing Support App

Three different types of buttons

► *Extensions*

- Run with elevated privileges
- Dynamically change the button text
- Dynamically change the icon
- Display Alert notification



Customizing Support App

Setup Mode

About Support⌘ I

✓ Configurator Mode⌘ C

⊗ Quit Support⌘ Q

Hi Kendall, Scott

Computer Name
MAC-KCJGJT4W22

macOS Tahoe
26.6

Last Reboot
1 day ago

166.09 GB Avail...

Password Info
Changed: 08/13/25
9 Days Left

Last Check-in
Wednesday
07:23 AM

IP Address
192.168.1.50
(Ethernet)

Battery
Normal
(Capacity: 97%)

Self Service+

Get Help

KB Articles

Service Catalog

Thrive

Expense Box

MyWorkDay

Product of IT Client Services for macOS

ExportEdit

Hi Kendall, Scott

Computer Name
MAC-KCJGJT4W22

macOS Tahoe
26.6

Last Reboot
1 day ago

54.3% Used

Password Info
Changed: 08/13/25
9 Days Left

Last Check-in
Wednesday
08:49 AM

IP Address
172.29.30.175
(VPN)

Battery
Normal
(Capacity: 97%)

Self Service+

Get Help

KB Articles

Service Catalog

Thrive

Expense Box

MyWorkDay

Product of IT Client Services for macOS

SettingsDone

Hi Kendall, Scott

<Item configurationSave

Item preview

Title

Item type

Button - Medium

✓ Button - Regular

Title

Button - Small

Extension

Subtitle

Action Type

Action

SF Symbol

Product of IT C

Settings

Basic Extension **Template**



Script Template (Layout)

The Setup (Script Header)

```
7
8  # Extension ID added for Support app 3.x config
9  extension_id="extensionID"
10
11 # Support App preference plist
12 preference_file_location="/Library/Preferences/nl.root3.support.plist"
13
14 # Start spinning indicator
15 defaults write "${preference_file_location}" "${extension_id}_loading" -bool true
16
17 # Replace value with placeholder while loading
18 defaults write "${preference_file_location}" "${extension_id}" -string "placeholder-text"
19
20 # Keep loading effect active specified time
21 sleep 0.25
22
```

Script Template (Layout)

Work Your Magic (Script Body)

```
23
24  # GET CREATIVE. DEFINE AND SOLVE A PROBLEM
25
26  # DETERMINE THE FOLLOWING
27  extension_status=""      # Text you want displayed on the tile beneath the extension title.
28  |   |   |   |   |   |   # You have 22-24 characters across 2 lines.
29  |   |   |   |   |   |   # It's important to account for all outcomes
30
31  # DETERMINE ANY OR NONE OF THE FOLLOWING
32  # Using these can avoid the need for a click script.
33  # https://github.com/root3nl/SupportApp/wiki/Extensions
34  action_type=""          # App | URL | Command | PrivilegedScript
35  action=""               # bundleID | URL to open | command run as user | command or script run with privileges
36  symbol=""              # Any SF Symbol string. https://developer.apple.com/sf-symbols/
37  alert_status=""         # true | false
38
```

Script Template (Layout)

Reveal to User (Script Footer)

```
40
41 # Action Types: App, URL, User Command, Privileged Script
42 defaults write "${preference_file_location}" "${extension_id}_action_type" -string "$action_type"
43
44 # Action: bundle ID of app to open, URL to open, or command to run when the tile is clicked.
45 defaults write "${preference_file_location}" "${extension_id}_action" -string "$action"
46
47 # Symbol: Any SF Symbol string
48 defaults write "${preference_file_location}" "${extension_id}_symbol" -string "$symbol"
49
50 # Set or unset the alert badge
51 defaults write "${preference_file_location}" "${extension_id}_alert" -bool "$alert_status"
52
53 # Write Corp Network Status to Support App preference plist
54 defaults write "${preference_file_location}" "${extension_id}" -string "$extension_status"
55
56 # Stop spinning indicator
57 defaults write "${preference_file_location}" "${extension_id}_loading" -bool false
58
59 exit
```

Modifications Using Extensions

- Networking example -





CREATING THE NETWORKING BUTTON:

- ▶ Title shows when dashboard opens
- ▶ Action calls Self Service Policy
- ▶ Display Icon (can be changed in Script!)
- ▶ Unique Extension Identifier
- ▶ On Appearance Action (Run the script)

Hi Kendall, Scott



< Item configuration Save

Item preview

 IP Address
Subtitle Example

Item type Extension

Title IP Address

Action Type URL

Action jamfselfservice://content?entity=pol

SF Symbol network

Extension Identifier NetworkInfo

On appear action /Library/Application Support/GiantEa

i This functionality is restricted to managed profiles for security reasons.

Product of IT Client Services for macOS

Settings

Extension Scripts (Network)

BEGINNING OF YOUR SCRIPT

- ▶ Set the location of the Support App directory
- ▶ Set the extensionID
 - They must match
 - NOT case sensitive
 - NO dot (.) in Extension Names)

```
supportAppDir="/Library/Preferences/nl.root3.support.plist"

# The extensionID variable will be used as a suffix for the keys we write to this plist, so they should be unique for
each extension you create.
extensionID="NetworkInfo"

# Set to true to enable color indicators (red/green circles) for good/bad network status
colorIndicator="true"

# Setup UI indicators
if [[ "$colorIndicator" == "true" ]]; then
    greenCircle="●" yellowCircle="●" redCircle="●"
else
    greenCircle="" yellowCircle="" redCircle=""
fi
```

Extension Scripts (Network)

SCRIPT BODY

- ▶ Start the spinner `_loading`
- ▶ Call your function(s)
- ▶ Set an alert `_alert`
- ▶ Change the verbiage `-string`
- ▶ Change the Icon `_symbol`
- ▶ Stop the Spinner `_loading`

```
# Set the loading key to true to trigger the loading animation  
# in the Support App while we retrieve the battery information
```

```
defaults write "$supportAppDir" "${extensionID}_loading" -bool true  
sleep 0.5
```

```
# Fetch info (modifies globals)
```

```
getNetworkStatus
```

```
# Write output to Support App preference plist
```

```
defaults write "$supportAppDir" "${extensionID}_alert" -bool "$ShowAlert"
```

```
# Write the text output to Support App preference plist
```

```
defaults write "$supportAppDir" "${extensionID}" -string "${nicStatu}"
```

```
# Show network icon based on connection type
```

```
defaults write "$supportAppDir" "${extensionID}_symbol" -string "${symbol}"
```

```
# turn off loading animation
```

```
defaults write "$supportAppDir" "${extensionID}_loading" -bool false
```

Extension Scripts (Network)

SCRIPT BODY

- ▶ Determine VPN App
- ▶ Obtain VPN IP address
- ▶ Create Button Text
- ▶ Change Icon Symbol

```
local ip vpn_bin

# 1. Check VPN (Highest Priority)
# Use -e (exists) and find the first match quickly
# Change this logic if you use a different VPN client or have a custom setup. This example checks
for bin in "/opt/cisco/secureclient/bin/vpn" "/opt/cisco/anyconnect/bin/vpn"; do
    if [[ -e "$bin" ]]; then
        vpn_bin="$bin"
        break
    fi
done

# If we found a VPN client, check if we're connected and get the IP address
# If a connection is found, retrieve the IP address and set the symbol to a lock to indicate VPN.
# This logic may need to be adjusted based on the specific output of your VPN client's status comm
if [[ -n "$vpn_bin" ]]; then
    ip=$(($vpn_bin stats 2>/dev/null | awk -F': ' '/Client Address \(IPv4\)/{print $2}' | xargs)
    if [[ -n "$ip" && "$ip" != "Not Available" ]]; then
        retval="${ip}\n(VPN)"
        symbol="lock.icloud"
        return
    fi
fi
```

Extension Scripts (Network)

SCRIPT BODY

- ▶ Determine Adapter
- ▶ Retrieve the IP address
- ▶ Create Button Text
- ▶ Change Icon Symbol

```
# 2. Check Ethernet (Prioritize wired)
# Find active services and filter for Ethernet-like names
# If found, retrieve the IP address and set the symbol to a desktop computer to indicate wired connection.
local eth_dev=$(networksetup -listnetworkserviceorder | awk -F 'Device: ' '/Ethernet|LAN/ {print $2}' | tr -d ' ')
for dev in ${(f)eth_dev}; do
    ip=$(ipconfig getifaddr "$dev" 2>/dev/null)
    if [[ -n "$ip" ]]; then
        retval="${ip}\n(Ethernet)"
        symbol="network"
        return
    fi
done

# 3. Check Wi-Fi
# Check for active Wi-Fi connection and retrieve the IP address. Set the symbol to a Wi-Fi icon if connected.
local wifi_dev=$(networksetup -listallhardwareports | awk '/Wi-Fi/{getline; print $2}')
ip=$(ipconfig getifaddr "$wifi_dev" 2>/dev/null)
if [[ -n "$ip" ]]; then
    retval="${ip}\n(Wi-Fi)"
    symbol="wifi"
    return
fi
```

Extension Scripts (Network)

SCRIPT FOOTER

- ▶ Set status symbols
- ▶ Set an alert `_alert`
- ▶ Change the verbiage `-string`
- ▶ Change the Icon `_symbol`
- ▶ Stop the Spinner `_loading`

```
# If the retval contains "No active", we consider that an alert state and set the text color to red.
# Otherwise, we set it to green.
# The symbol is set based on the type of connection found (VPN, Ethernet, Wi-Fi, or no connection)

nicStatus="${greenCircle} ${retval}"
if [[ "$retval" == "No active"* ]]; then
    ShowAlert=true
    nicStatus="${redCircle} ${retval}"
elif [[ "$retval" == *"169.254"* ]]; then
    ShowAlert=true
    nicStatus="${yellowCircle} ${retval}"
fi

# Write output to Support App preference plist
defaults write "$supportAppDir" "${extensionID}_alert" -bool "$ShowAlert"

# Write the text output to Support App preference plist
defaults write "$supportAppDir" "${extensionID}" -string "${nicStatus}"

# Show network icon based on connection type
defaults write "$supportAppDir" "${extensionID}_symbol" -string "${symbol}"

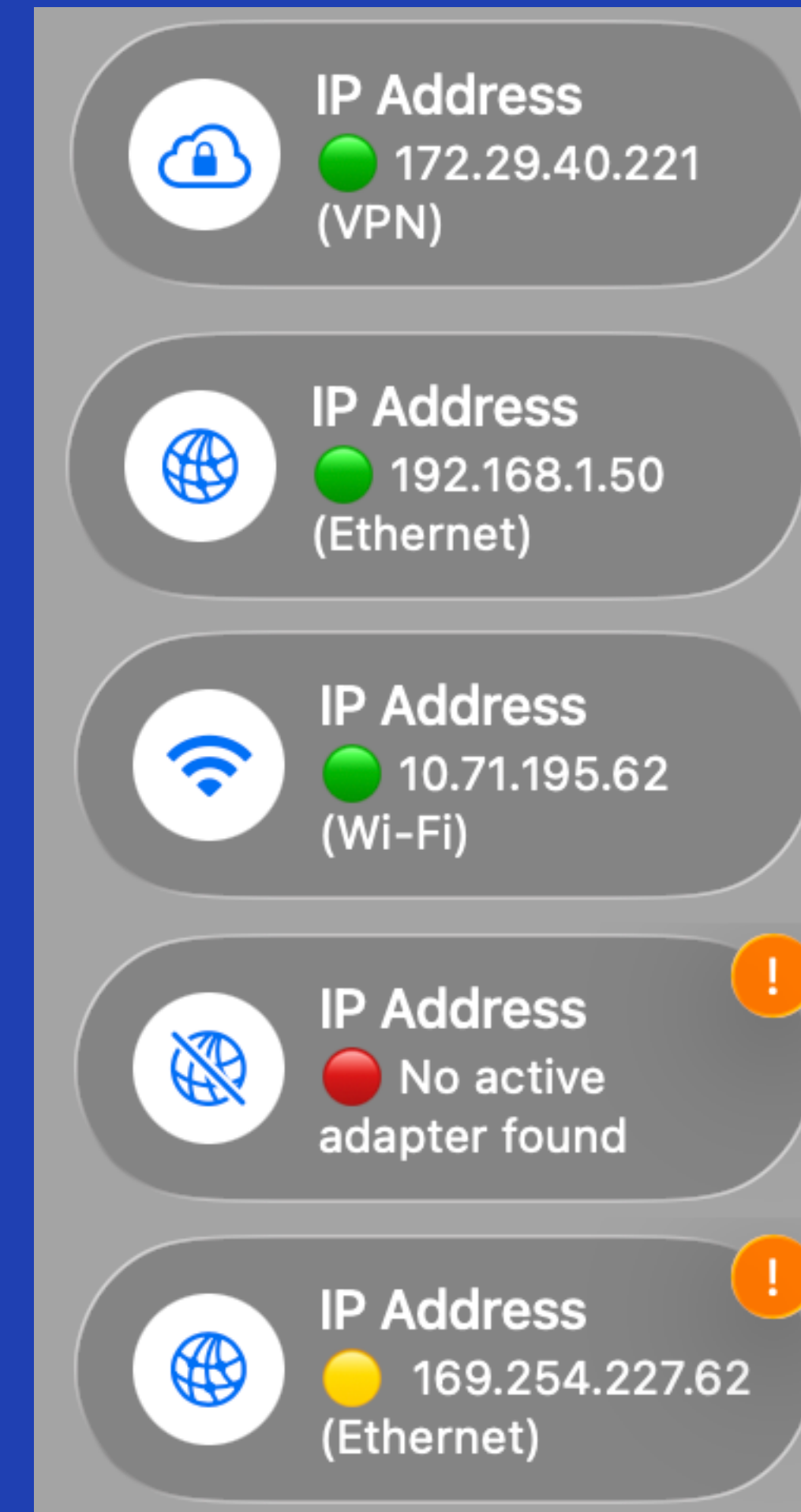
# turn off loading animation
defaults write "$supportAppDir" "${extensionID}_loading" -bool false
```

Extension Scripts (Network)

FIVE POSSIBLE OUTPUTS USING THIS SCRIPT:

- ▶ VPN Connection with *lock.icloud* icon
- ▶ Ethernet with *network* icon
- ▶ Wi-Fi with *wifi* icon
- ▶ No Adapters with alert & *network.slash* icon
- ▶ Self Assigned IP with alert & *network* icon

*The Icons used are from the SF library



Modifications Using Extensions

- Passwords example -





CREATING THE PASSWORD BUTTON:

- ▶ Title shows when dashboard opens
- ▶ Action redirects to Microsoft Change Password URL
- ▶ Unique Extension Identifier
- ▶ On Appearance Action (Run the script)

Hi Kendall, Scott



< Item configuration

Save

Item preview

 Password Info
Subtitle Example

Item type

Extension

Title

Password Info

Action Type

URL

Action

<https://mysignins.microsoft.com/sec>

SF Symbol

person.badge.key.fill

Extension Identifier

GetPasswordAge

On appear action

/Library/Application Support/GiantEa

 This functionality is restricted to managed profiles for security reasons.

Product of IT Client Services for macOS

Settings

Extension Script (Password)

BEGINNING OF YOUR SCRIPT

- ▶ Do not use \$HOME to get Users Home Directory
 - Due to running as privileged environment
- ▶ Set the location of the Support App directory
- ▶ Set the extensionID (again...they must match)

```
# NOTE: you cannot use the $HOME variable in a support.app extension script because the extension runs as root
# and $HOME will resolve to /var/root which is not what we want.
LOGGED_IN_USER=$( scutil <<< "show State:/Users/ConsoleUser" | awk '/Name :/ && ! /loginwindow/ { print $3 }' )
USER_DIR=$( dscl . -read /Users/${LOGGED_IN_USER} NFSHomeDirectory | awk '{ print $2 }' )

# Location of the com.GiantEagleEntra.plist for the current user
localXMLFile="$USER_DIR/Library/Application Support/com.GiantEagleEntra.plist"

# Location of the Support App preference plist where we will write the password age and days left until expiration
supportAppDir="/Library/Preferences/nl.root3.support.plist"

# support.app Extension ID...this MUST match the extension ID you set in the button
extensionID="GetPasswordAge"
```

Hi Kendall, Scott

Item configuration

Save

Item preview

Password Info
Subtitle Example

Item type: Extension

Title: Password Info

Action Type: URL

Action: https://mysignins.microsoft.com/sec

SF Symbol: person.badge.key.fill

Extension Identifier: GetPasswordAge

On appear action: /Library/Application Support/GiantEagleEntra.plist

This functionality is restricted to managed profiles for security reasons.

Product of IT Client Services for macOS

Settings

Password Script

SCRIPT BODY

- ▶ Get User's Home Directory
- ▶ Set ENV Variables
- ▶ Start the spinner
- ▶ Retrieve Password Info
- ▶ Change the button title/subtitle
- ▶ Show an alert if password limit close
- ▶ Stop the spinner

```
LOGGED_IN_USER=$( scutil <<< "show State:/Users/ConsoleUser" | awk '/Name :/ && ! /loginwindow/ { print $3 }' )
USER_DIR=$( dscl . -read /Users/${LOGGED_IN_USER} NFSHomeDirectory | awk '{ print $2 }' )
```

```
localXMLFile="$USER_DIR/Library/Application Support/com.GiantEagleEntra.plist"
```

```
supportAppDir="/Library/Preferences/nl.root3.support.plist"
```

```
extensionID="GetPasswordAge"
```

```
passwordLimit=365
```

```
notificationLimit=14
```

```
defaults write "$supportAppDir" "${extensionID}_loading" -bool true
sleep .5
```

```
if [[ -e $localXMLFile ]]; then
    PasswordAge=$(defaults read "$localXMLFile" "PasswordAge")
    LastPasswordChange=$(defaults read "$localXMLFile" "PasswordLastChanged")
else
    PasswordAge=0
    LastPasswordChange=$(date -u +"%Y-%m-%dT%H:%M:%SZ")
fi
dayleft=$((passwordLimit - PasswordAge))

LastPasswordChangeDate=$(date -j -f "%Y-%m-%dT%H:%M:%SZ" $LastPasswordChange +"%X")
```

```
defaults write "$supportAppDir" "${extensionID}" -string "Changed: ${LastPasswordChangeDate}\n${dayleft} Days Left"
```

```
showAlert=false
if [[ $dayleft -le $notificationLimit ]]; then
    showAlert=true
fi
defaults write "$supportAppDir" "${extensionID}_alert" -bool $showAlert
```

```
defaults write "$supportAppDir" "${extensionID}_loading" -bool false
```

Modifications Using Extensions

- Battery Charge / Health -



Extension Script (Battery)

SCRIPT BODY

- ▶ Determine Desktop/Laptop
- ▶ Get current Charge
- ▶ Set Icon based on Charge

```
function getBatteryHealth() {  
  
    # 1. Check if it's a laptop by checking if battery registry data exists  
    modelName=$(system_profiler SPHardwareDataType 2>/dev/null | awk -F': ' '/Model Name/ {print $2}')
```

```
    if [[ $modelName != MacBook* ]]; then  
        retval="Not A Laptop"  
        symbol="desktopcomputer"  
        return  
    fi
```

```
    # 2. Fast extraction of current charge using ioreg  
    powerData=$(ioreg -n AppleSmartBattery -r 2>/dev/null)  
    local currentCharge=100  
    if [[ "$powerData" =~ '"CurrentCapacity" = ([0-9]+)' ]]; then  
        currentCharge=${match[1]}  
    fi
```

```
    # 3. Map charge to SF Symbols  
    symbol="battery.0"  
    if (( currentCharge > 74 )); then symbol="battery.100"  
    elif (( currentCharge > 49 )); then symbol="battery.75"  
    elif (( currentCharge > 24 )); then symbol="battery.50"  
    elif (( currentCharge >= 0 )); then symbol="battery.25"  
    fi
```

```
}
```

Extension Script (Battery)

SCRIPT BODY

- ▶ Determine Battery Health
- ▶ Calculate Battery Life
- ▶ Set status / Capacity Indicator

```
# 5. Extract Health Condition using Zsh native regex (No grep/awk/xargs needed)
HealthCondition="Normal"
if [[ "$powerData" =~ '"PermanentFailureStatus" = ([0-9]+)' && "${match[1]}" != "0" ]]; then
    HealthCondition="Service Battery"
fi

# 6. Check architecture natively using Zsh parameters

if [[ "$(sysctl -n hw.optional.arm64 2>/dev/null)" == "1" ]]; then
    # Apple Silicon: Calculate Maximum Capacity (State of Health)
    # Formula: (AppleRawMaxCapacity / DesignCapacity) * 100

    [[ "$powerData" =~ '"AppleRawMaxCapacity" = ([0-9]+)' ]] && rawMax=${match[1]}
    [[ "$powerData" =~ '"DesignCapacity" = ([0-9]+)' ]] && designCap=${match[1]}

    if [[ -n "$rawMax" && -n "$designCap" ]]; then
        (( maxCapacity = (rawMax * 100) / designCap ))
    fi
fi

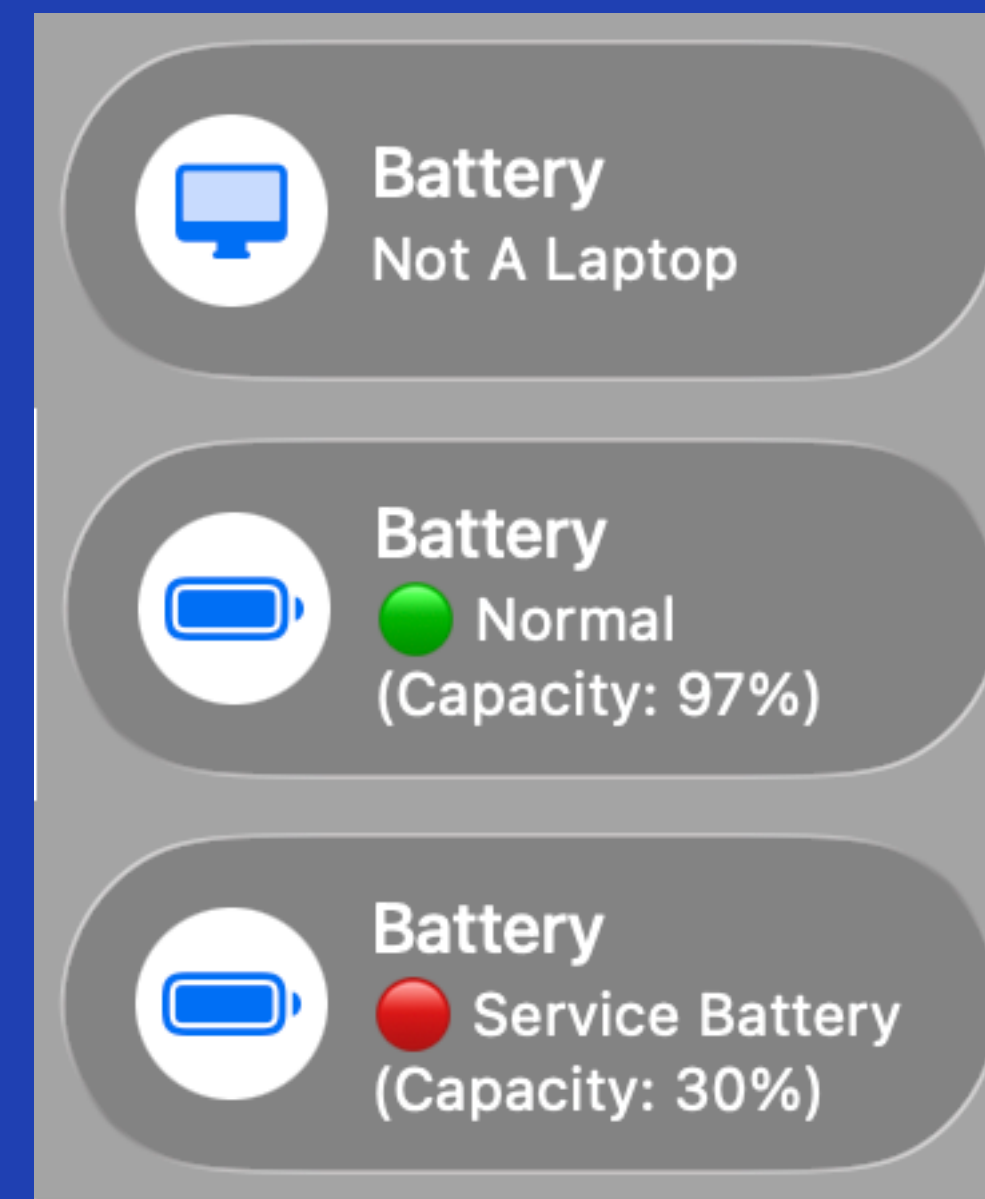
# Determine color circle based on maximum capacity calculations
if (( maxCapacity > 89 )); then retval=$greenCircle
elif (( maxCapacity > 79 )); then retval=$yellowCircle
else
    retval=$redCircle
fi
retval+="${HealthCondition}\n(Capacity: ${maxCapacity}%)"
else
    # Intel Mac default fallback
    retval="${greenCircle}${HealthCondition}\n"
fi
```

Extension Script (Battery)

THREE POSSIBLE OUTPUTS USING THIS SCRIPT:

- ▶ Desktop (no Battery Info) & Desktop Icon*
- ▶ Battery (Condition & Capacity)
- ▶ Battery (Failed & Capacity)

*The Icons used are from the SF library

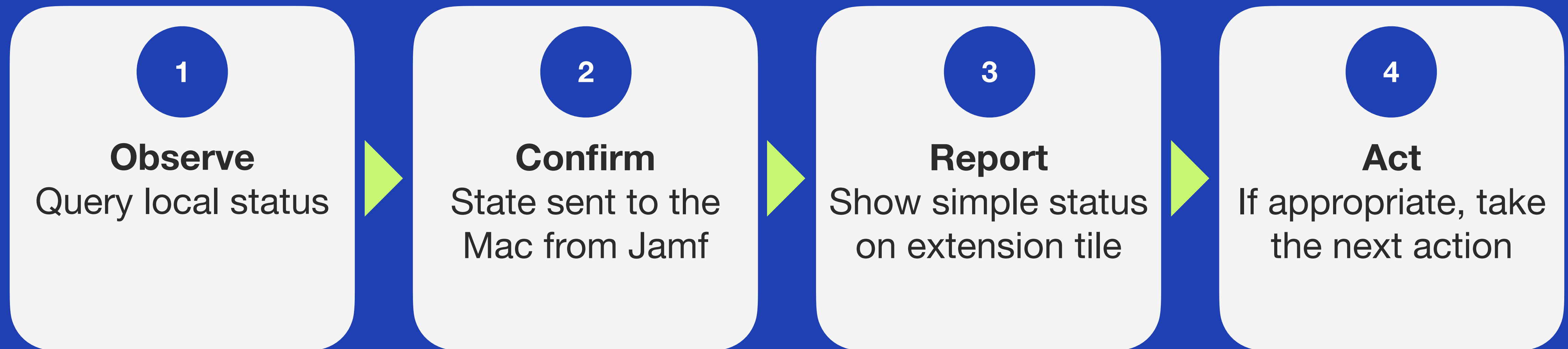


Modifications Using Extensions

- Registration Example -



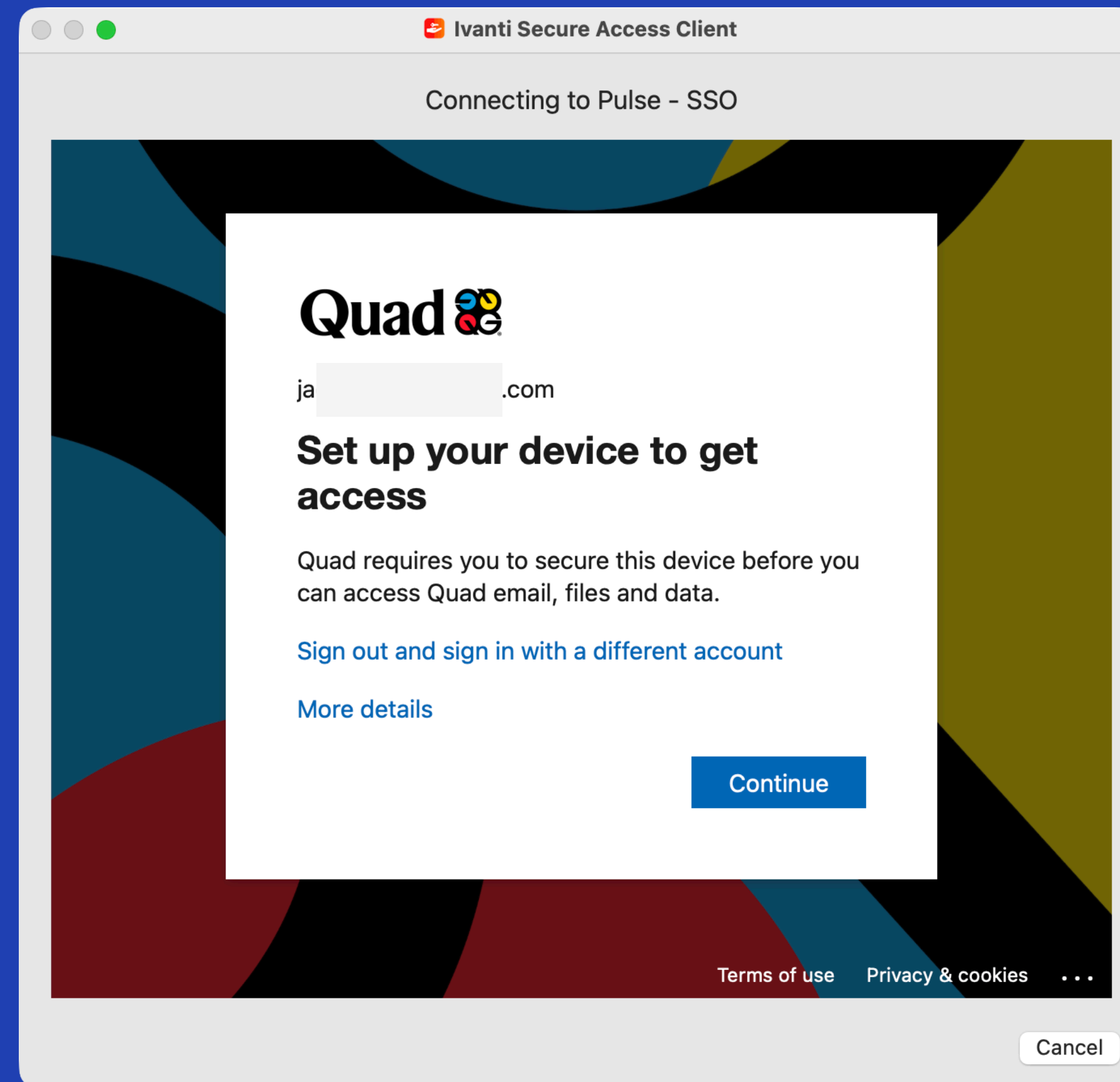
The Model



Registration

PROBLEM

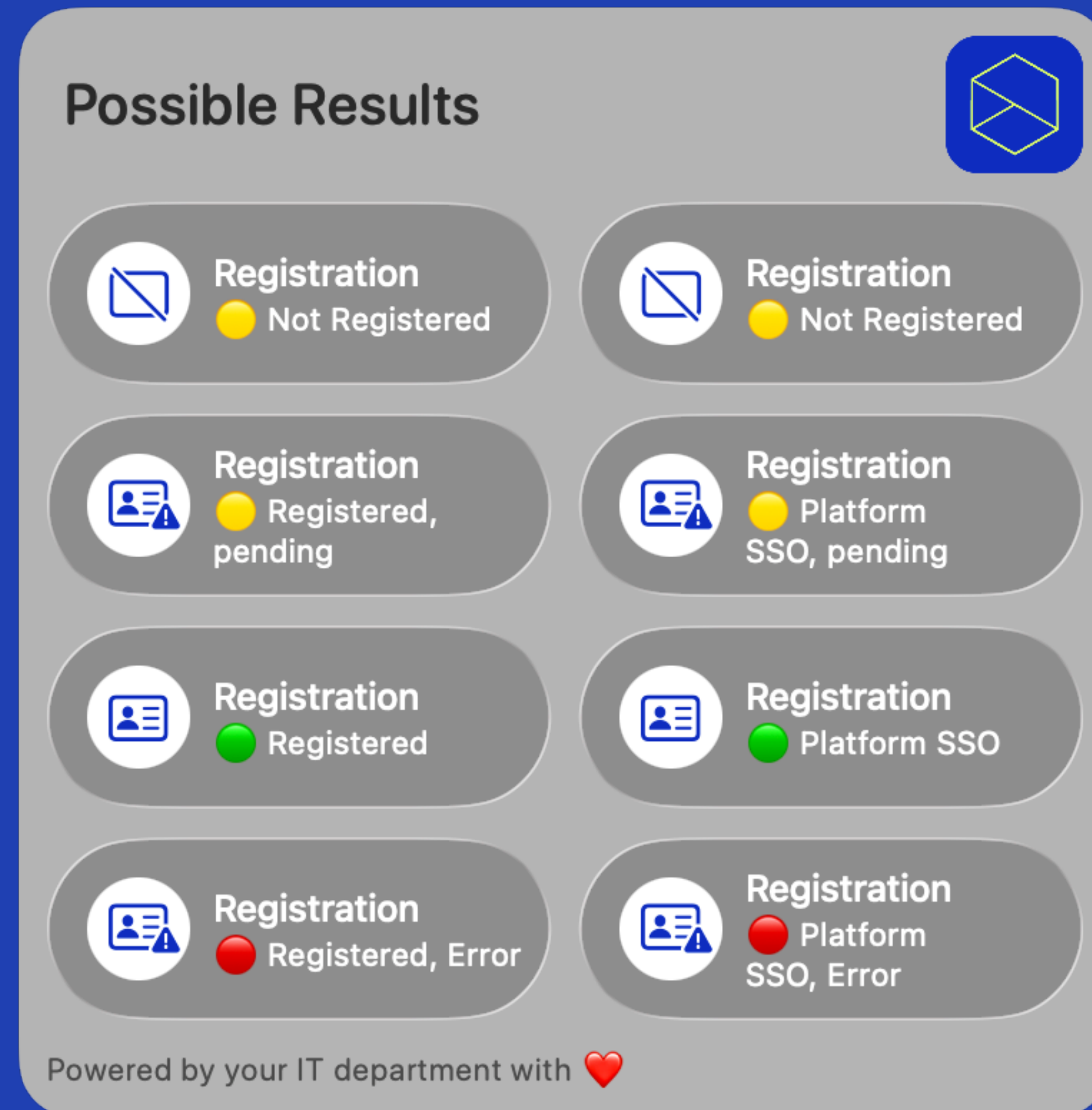
- ▶ Users unsure of registration status
- ▶ Next steps for users weren't always clear
- ▶ Help Desk
 - Uncomfortable with Mac support
 - Reference old documentation



Registration

SOLUTION

- ▶ Tile to show status and take action
- ▶ Account for Platform SSO and Jamf Device Compliance
- ▶ Local profile signal confirms Jamf state



Registration

SCRIPT HIGHLIGHTS

- ▶ Differentiate between pSSO and Jamf Device Compliance
- ▶ Adjust for device ID in Secure Enclave vs Keychain
- ▶ If / Else logic to determine status

```
76
77
78 # Check jamfAAD plist for AAD ID
79 AAD_ID=$(defaults read "$jamfaad_plist" have_an_Azure_id 2> /dev/null)
80
81 #Check if registered via PSS0/SS0e
82 ssoStatus=$(/bin/launchctl asuser $loggedInUserID "${jamfCA}" getPSS0Status | /usr/bin/sed -E 's/AnyHashable\\(|\\)/g' | /usr/bin/tr ' ,' '\n')
83
84 # Check Apple Platform SSO registration
85 platformStatus=$(su "$loggedInUser" -c "app-sso platform -s" 2>/dev/null | awk '/registration/ {gsub(/,/,""); print $3}')
86
87 # Check for proxy configuration profile key that is scoped to the registered smart group
88 # Smart Group Criteria: Device Compliance Status - Registration Status is Registered
89 [[ $(defaults read "${compliancePlistPath}" "registeredGroup" 2> /dev/null) ]] && registeredGroup="Yes" || registeredGroup="No"
90
91
```

Registration

DEPENDENCIES

- ▶ Smart group
- ▶ Configuration profile with custom setting payload
- ▶ Closes the loop; approximates timing of info going to Microsoft Entra

Computers : Smart computer groups

← Device Compliance Registered

Computer Group Criteria Reports

AND/OR	CRITERIA	OPERATOR	VALUE
▼	Device Compliance Integration - Registration Status	is ▼	Registered ⋮

Preference Domain

com.jnuc.devicecompliance

The name of the preference domain (com.company.application)

Upload File

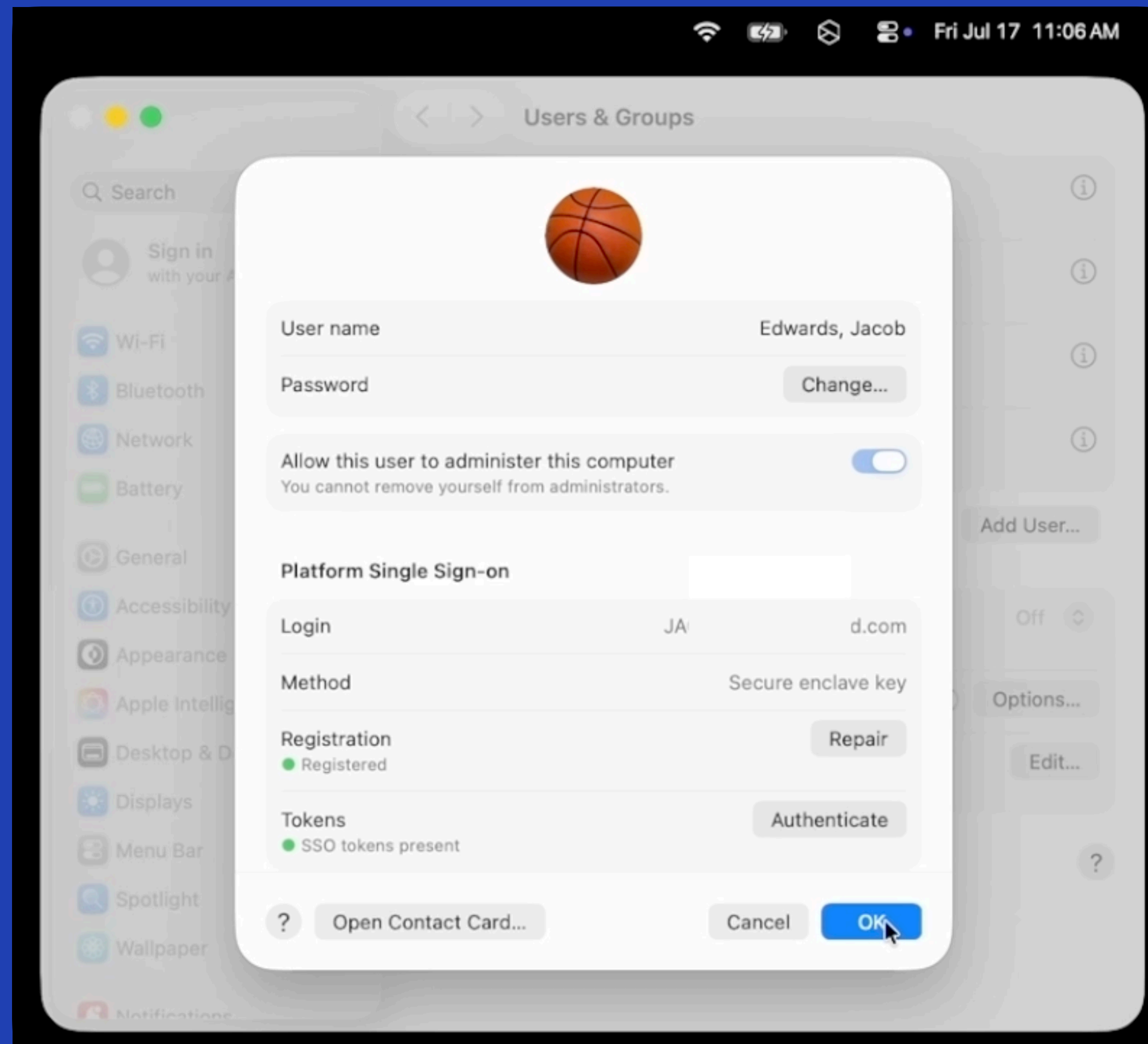
PLIST file containing key value pairs for settings in the specified domain

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
 <dict>
 <key>registeredGroup</key>
 <true/>
 </dict>
</plist>

Registration

SCRIPT ACTIONS

- ▶ Start desired registration type
- ▶ Move registration process along
- ▶ For Platform SSO, open registration details



Modifications Using Extensions

- Compliance Example -



Compliance

PROBLEM

- ▶ Users may lose device compliance
 - Miss an update deadline
 - Enforcement not working as intended
 - Mac offline for extended period

☐	Name ↑↓	Enabled	OS	Version	Join type	Owner	MDM	Security settings m...	Compliant	Registered ↑↓	Activity ↑↓
☐	 DemoMac	✔ Yes	MacOS	26.5.2	Microsoft Entra joined	Edwards, Jacob	None	N/A	❗ No	7/5/2026, 8:59 AM	7/5/2026, 8:59 ...

Compliance

SOLUTION

- ▶ Tile to show status and report details

NOTE

- ▶ Organizations can have a variety of compliance requirements

Possible Results



-  **Compliance**
● Not Registered
-  **Compliance**
● Compliant, pending
-  **Compliance**
● Compliant
-  **Compliance**
● Not Compliant

Powered by your IT department with ❤️

Compliance

SCRIPT HIGHLIGHTS

- ▶ Slight delay so registration status is calculated first
- ▶ Check for profiles
 - Compliance status
 - Compliant group membership
- ▶ Return the status

```
65
66 # Keep loading effect active for at least the specified sleep time
67 sleep 0.2
68
69 registrationStatus=$(defaults read "${preference_file_location}" "registration" 2> /dev/null)
70 # Wait for registration tile to be determined before checking compliance status
71 until [[ "$registrationStatus" != "Checking status" ]] && [[ "$registrationStatus" != "" ]]; do
72     sleep 0.1
73     registrationStatus=$(defaults read "${preference_file_location}" "registration" 2> /dev/null)
74 done
75
```

```
75
76 checkComplianceStatus() {
77     # Check for profile indicating that all compliance requirements are met.
78     [[ $(defaults read "${compliancePlistPath}" "compliance" 2> /dev/null) ]] && complianceReqs="Yes" || complianceReqs="No"
79     # Check for profile indicating that Jamf shows the Device Compliance Status – Compliance Status is Compliant
80     [[ $(defaults read "${compliancePlistPath}" "compliantGroup" 2> /dev/null) ]] && compliantGroup="Yes" || compliantGroup="No"
81
```

Compliance

DEPENDENCIES

- ▶ Meets compliance requirements
 - Set some key to true
 - Scope to the compliant group designated in Jamf Pro Settings > Device Compliance

Preference Domain
The name of the preference domain (com.company.application)

com.jnuc.devicecompliance

Upload File
PLIST file containing key value pairs for settings in the specified domain

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
  <dict>
    <key>compliance</key>
    <true/>
  </dict>
</plist>
```

Settings : Global

← **Device Compliance**

Configuration status ☒
Use the switch to enable or disable the connection.

✔ Connection verification status: Success

Platform Select platform type to configure.

✔ macOS

Compliance Group Smart computer group Jamf Pro will use to calculate device compliance.

Device Compliance - Meets Compliance

Applicable Group
Smart group containing all computers Jamf Pro uses to send a compliance status to Microsoft Intune. This also makes the Register button available in Self Service.

Device Compliance - Applicable

Compliance

DEPENDENCIES

- ▶ Meets compliance requirements
 - Set some key to true
 - Scope to the compliant group designated in Jamf Pro Settings > Device Compliance
- ▶ Status sent to Entra
 - Set some key to true
 - Scope to group with criteria of Device Compliance Integration - Compliance Status is Compliant

Preference Domain

The name of the preference domain (com.company.application)

com.jnuc.devicecompliance

Upload File

PLIST file containing key value pairs for settings in the specified domain

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
 <dict>
 <key>compliantGroup</key>
 <true/>
 </dict>
</plist>

Computers : Smart computer groups

←

Device Compliance - Compliant Status

Computer Group

Criteria

Reports

AND/OR	CRITERIA	OPERATOR	VALUE
▼	Device Compliance Integration - Compliance Status	is ▼	Compliant ...

Compliance

DEPENDENCIES

- ▶ Clearly defined compliance component requirements
- ▶ Smart group and configuration profile for each component

← Device Compliance Tahoe 26.x

Computer Group Criteria Reports

AND/OR	CRITERIA	OPERATOR	VALUE			
(	Operating System Version	greater than or equal	26.5.1	...	▼	Delete
and	Operating System Version	less than	27.0.0	...	)	Delete

Computers : Configuration profiles

← Device Compliance macOS

Options Scope

☐ Show in Jamf Pro Dashboard

Search...

General

Application & Custom Settings
1 payload configured

Upload

The name of the preference domain (com.company.application)

Upload File

PLIST file containing key value pairs for settings in the specified domain

<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd"><plist version="1.0"><dict><key>macos_minimum</key><true/></dict></plist>

Compliance

DEPENDENCIES

- ▶ Final Profile
 - Overall Compliance
 - Entra confirmations
 - Components

PROFILE

com.jnuc.devicecompliance.plist

XML SOURCE

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>compliance</key>
  <true/>
  <key>registeredGroup</key>
  <true/>
  <key>compliantGroup</key>
  <true/>
  <key>filevault</key>
  <true/>
  <key>macos_minimum</key>
  <true/>
</dict>
</plist>
```

Compliance

REPORT

- ▶ Output granular details in swiftDialog
- ▶ Add any definable requirement

This Mac **does not meet** compliance requirements.

This status may not reflect recent changes. If you are having problems with device compliance, please contact the Help Desk.

Compliance Report

Mac Name:

DemoMac

Registration:

Platform SSO

Compliance Status:

No

macOS Minimum:

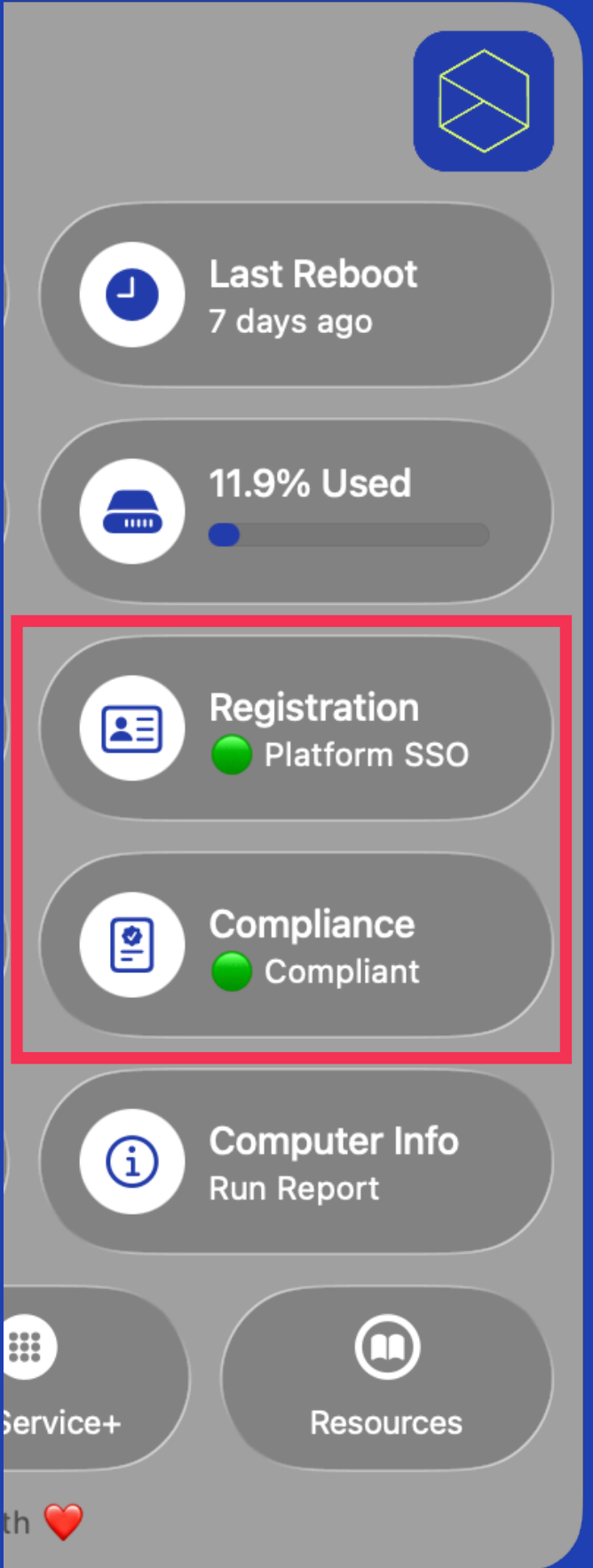
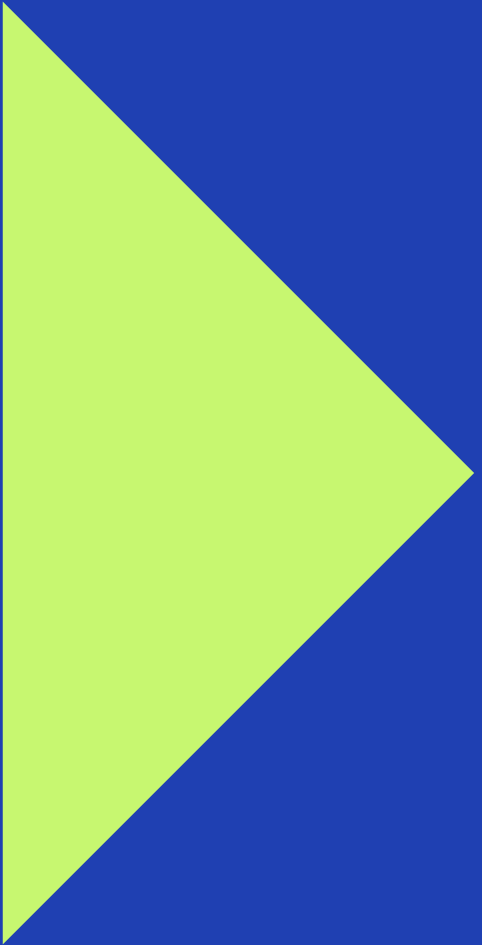
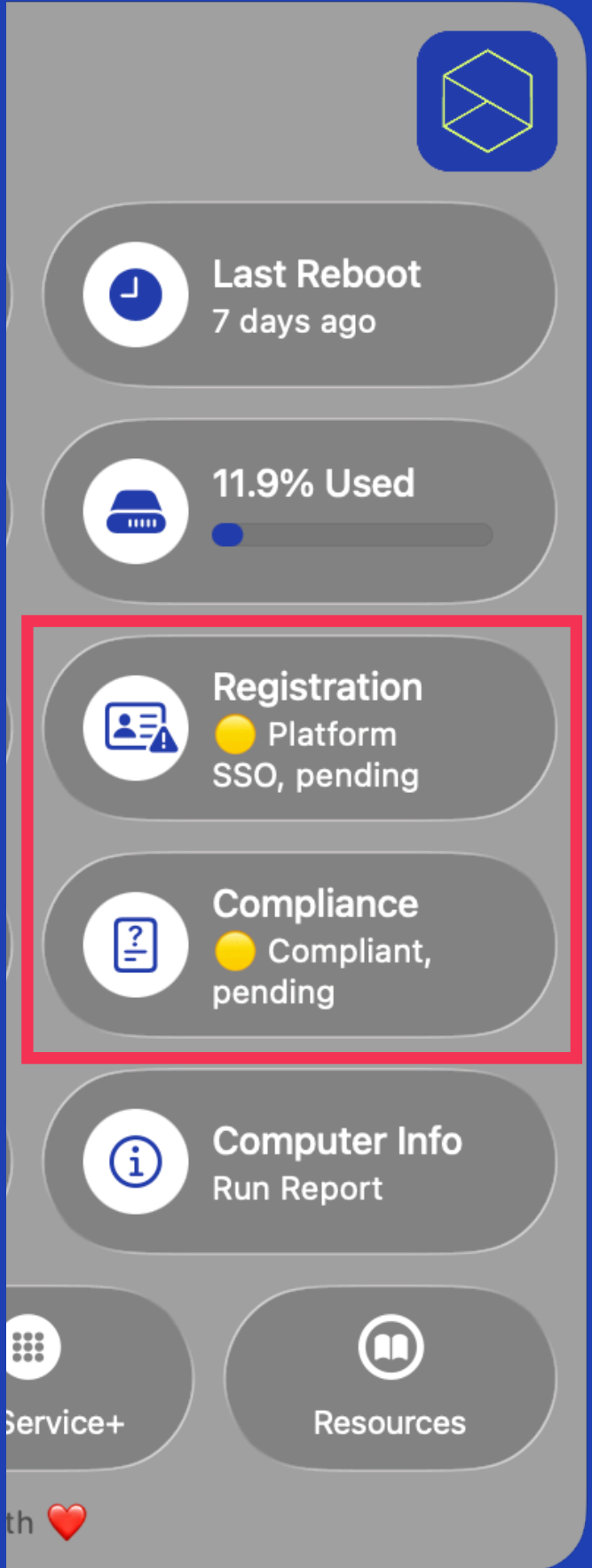
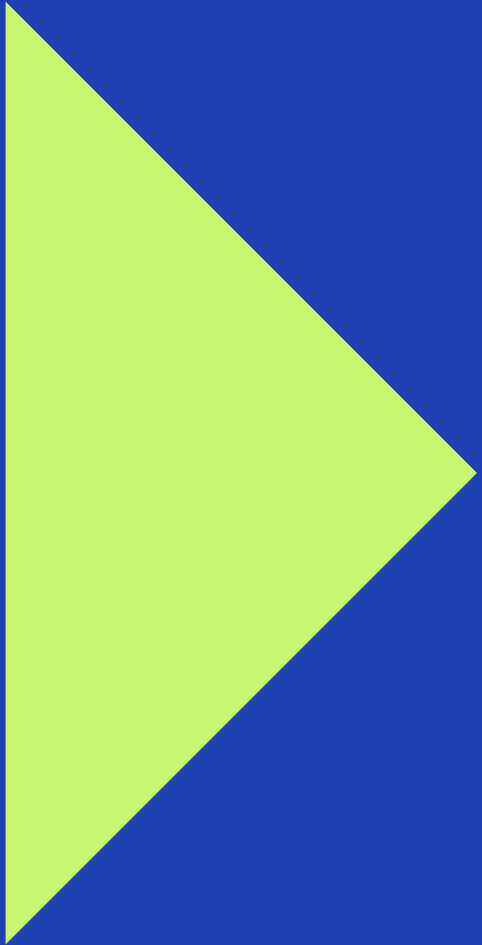
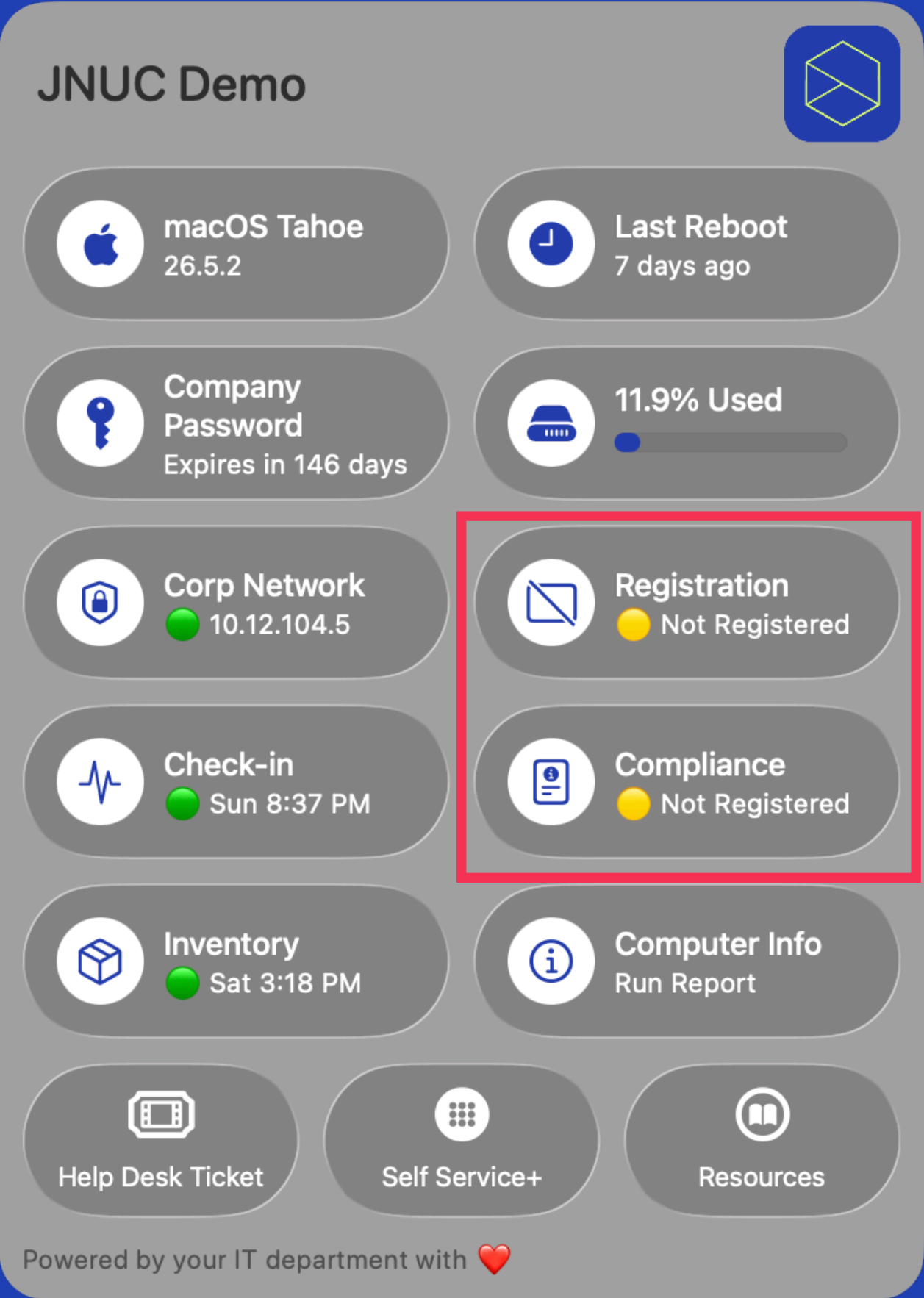
No

FileVault:

Yes

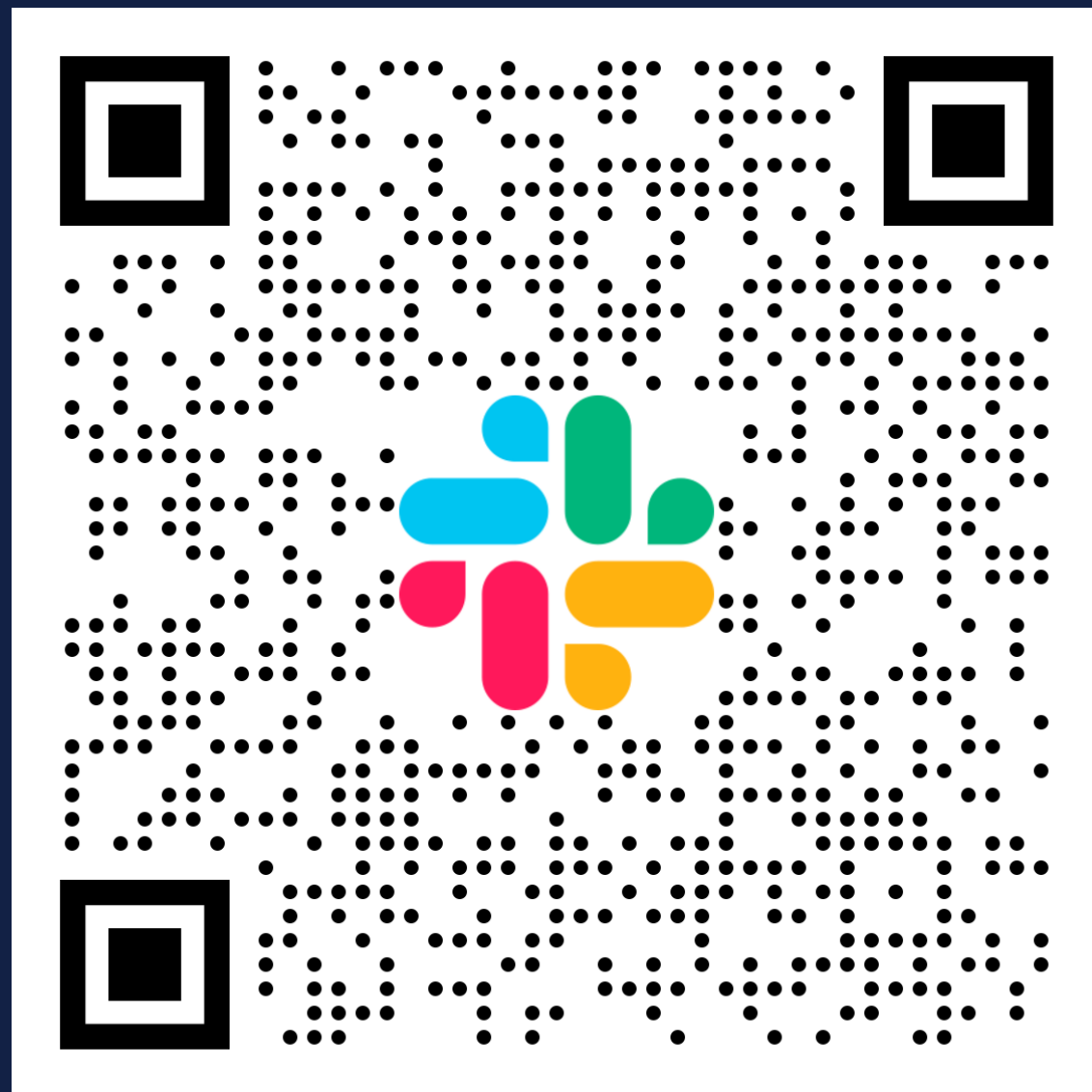
OK

Bringing It All Together



Join the Conversation

#support-app



#Slack IDs

@jacobedwards

@ScottKendall

Other Ideas/Examples

- ▶ Last Inventory
- ▶ Last Check-in
- ▶ Privilege elevation
- ▶ Lifecycle/Replacement notice
- ▶ mSCP Compliance
- ▶ Health check
- ▶ App Auto-Patch - Andrew Spokes

We are hosting a Braindate

Scan the code to join and continue the conversation



- ▶ Login and use the code
- ▶ Search for my name in the Topic Marketplace
- ▶ Book your spot
- ▶ Limited seating available. If the session is full, please stop by the Braindate Lounge and the team we'll do their best to accommodate.
- ▶ Let's connect again in the Braindate Lounge!



Special Thanks

- ▶ Root3 - creating and publishing Support App
- ▶ Ben Whitis, Jamf - sharing knowledge about SSOe / pSSO

THE REPOS



<https://github.com/root3nl/Support App>



<https://github.com/ScottEKendall/JNUC2026>

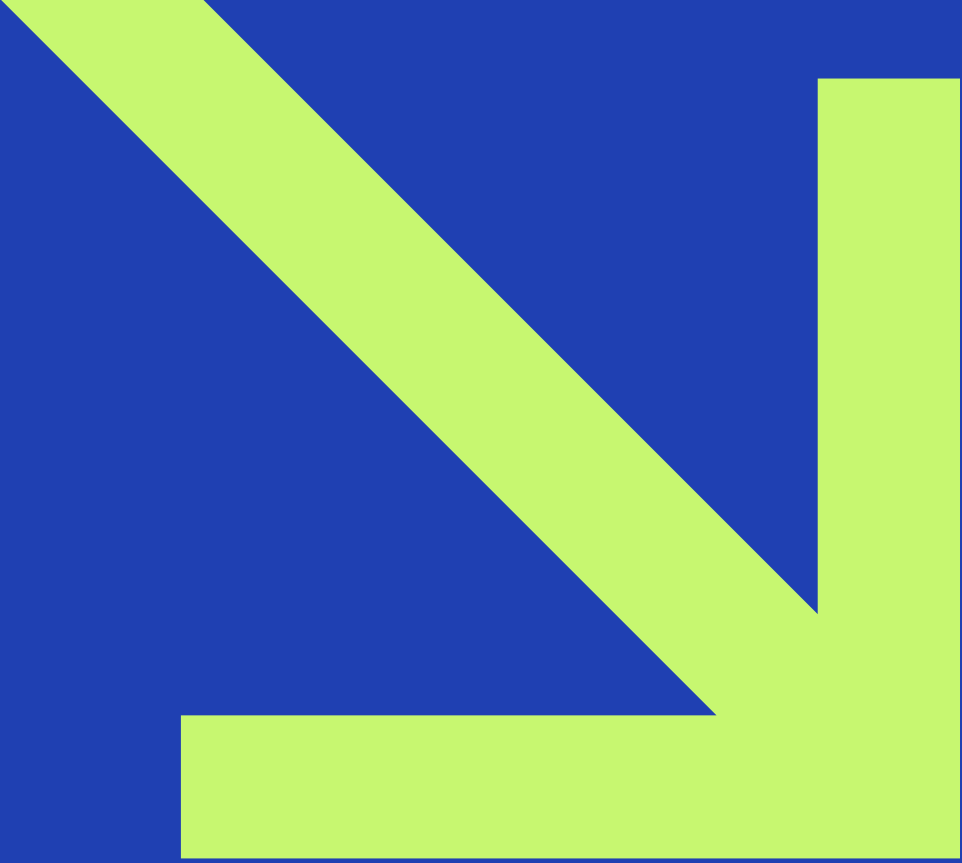


<https://github.com/jaedwards24/JNUC2026>





QUESTIONS?
YOUR TURN!



THANK
YOU